

SCS overview

Terms

- ◆ SCS stands for Simple Communication System
- ◆ AF stands for Application Framework.

Disclaimers

- ◆ This is a SCS presentation.
- ◆ It is not meant to serve or be a substitute for SCS protocol documentation.
- ◆ This presentation is polluted with examples of how Autonomous Mining Truck uses the SCS.

Disclaimers

- ◆ This is a SCS presentation.
- ◆ It is not meant to serve or be a substitute for SCS protocol documentation.
- ◆ This presentation is polluted with examples of how Autonomous Mining Truck uses the SCS.
 - ◆ AMT application code does not use SCS directly
 - ◆ AMT application code uses AF
 - ◆ AF in turn uses SCS

Productivity

AMT application Code



AIS Application Framework

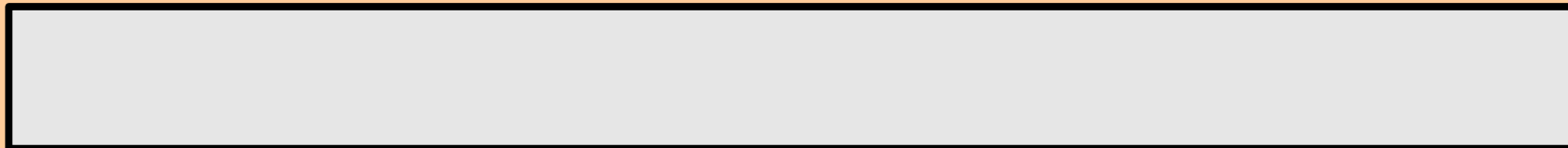


SCS

Code base

- ◆ IM = Integration Milestone
- ◆ This presentation has been prepared based on AMT IM4.1
- ◆ AHS uses now IM4.5

AF



Layer 7: Application layer

- ◆ **marshaling/serialization of objects**
- ◆ **[optional] compression**

Layer 6: Presentation layer

SCS

- ◆ **connections management**
- ◆ **duplex and simplex operation**

Layer 5: Session layer

- ◆ **segmentation/desegmentation**
- ◆ **error checking and old data discarding**
- ◆ **no retransmission*****
- ◆ **flow control*****

Layer 4: Transport layer

AF & simplex operation

SCS

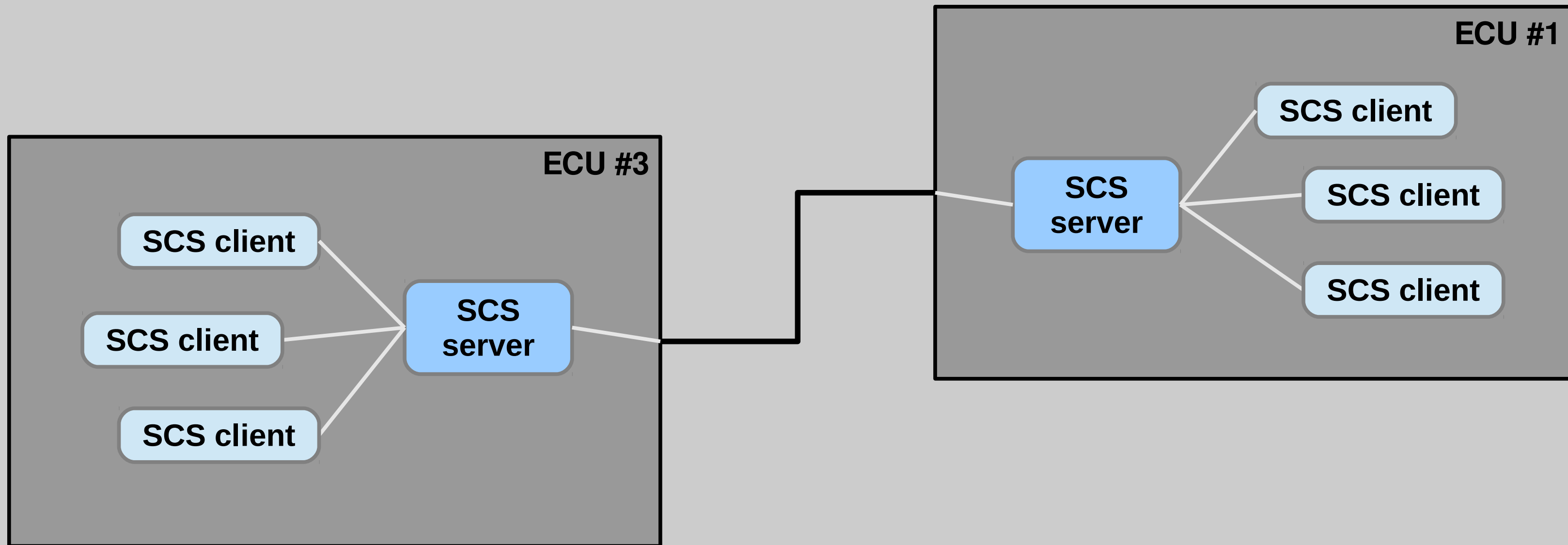
- ◆ connections management
- ◆ duplex and **simplex** operation

Layer 5: Session layer

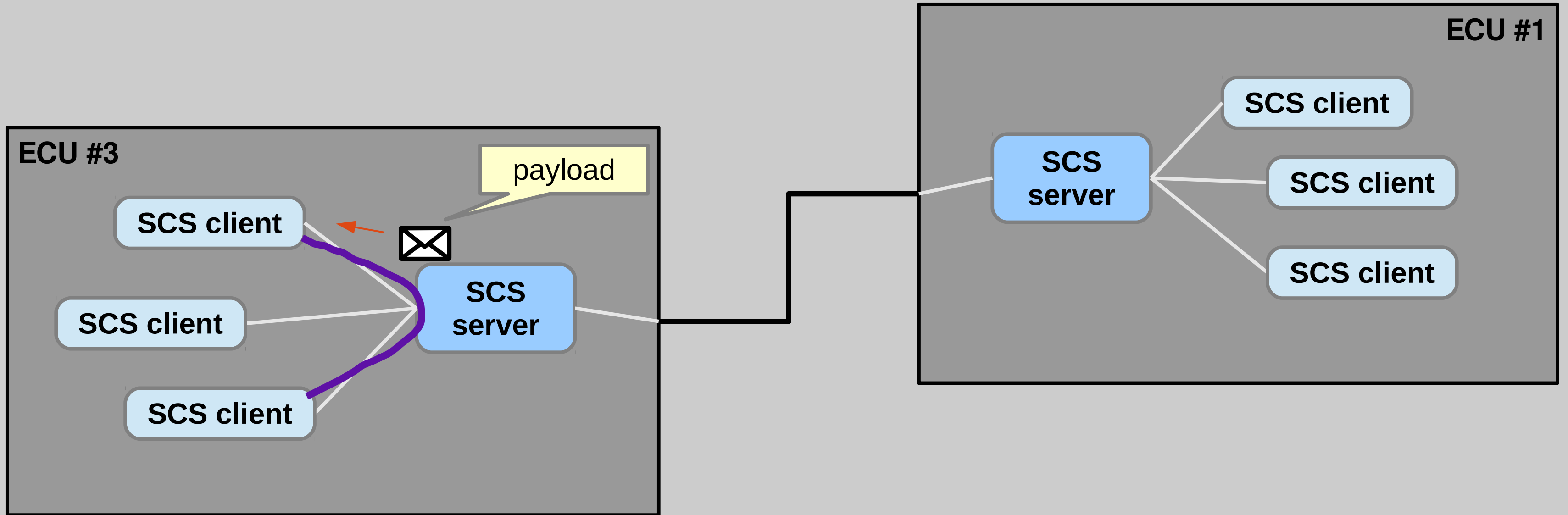
- ◆ AMT AF uses **SCS** in **simplex** mode.
- ◆ Examples presented assume simplex mode.

Layer 4: Tran. layer

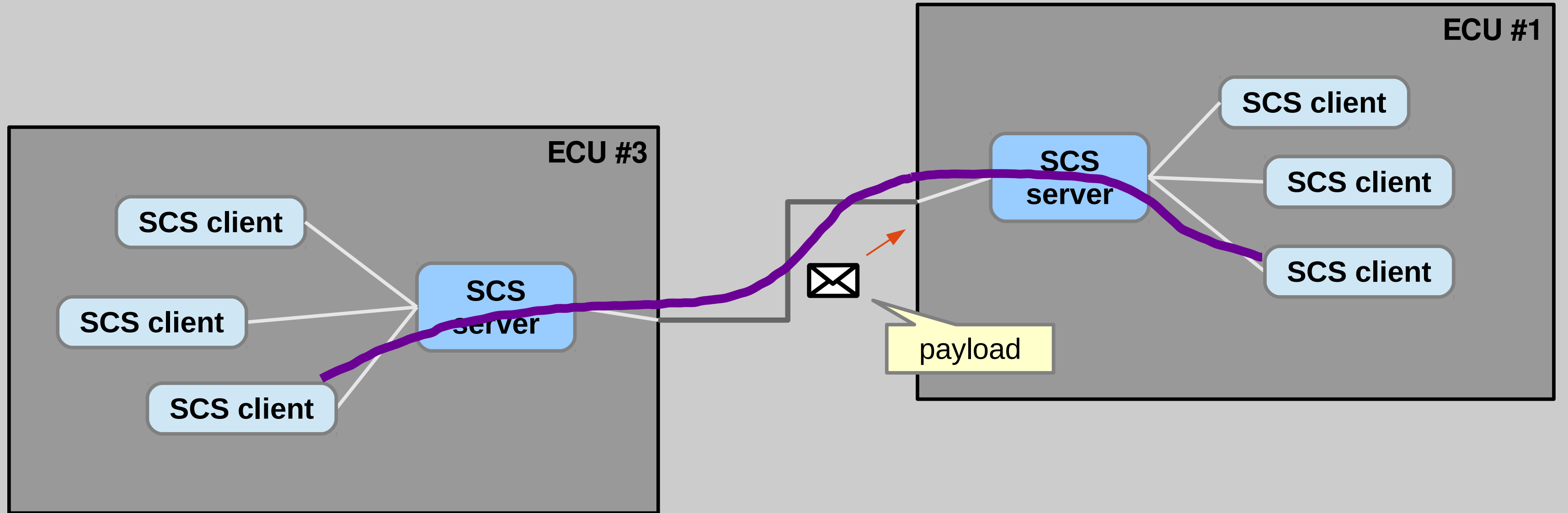
server(s) & clients



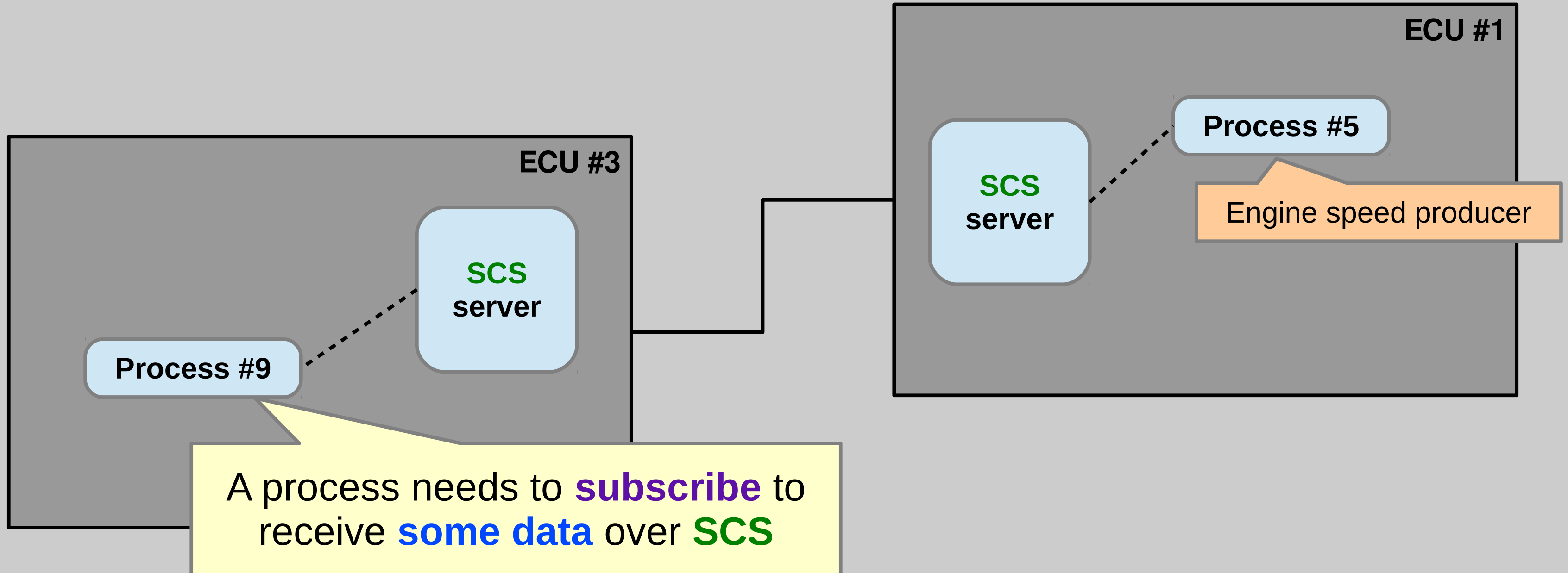
local to local client



local to remote client



subscription based



no consumers

ECU #2

ECU #6

ECU #3

ECU #1

Process #5

SCS
server

Engine speed producer

With no consumers i.e., active subscriptions
engine speed is dropped by **SCS** server



no remote consumers

ECU #2

ECU #6

ECU #3

ECU #1

Process #5

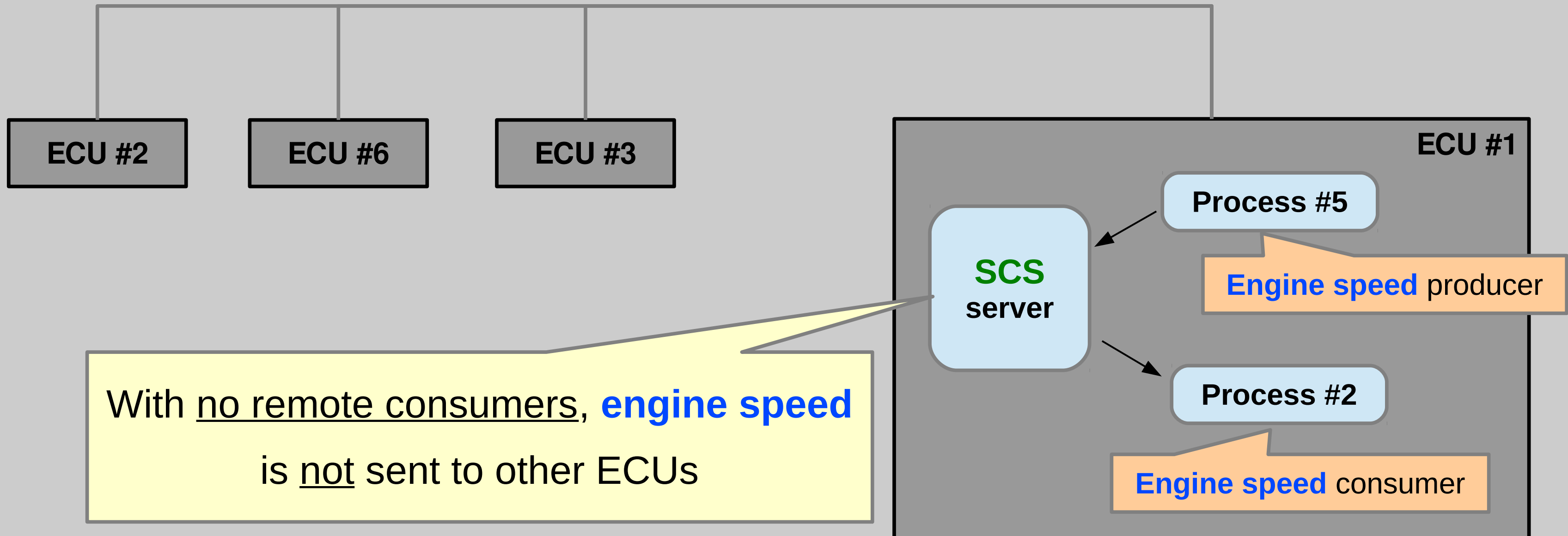
Engine speed producer

SCS
server

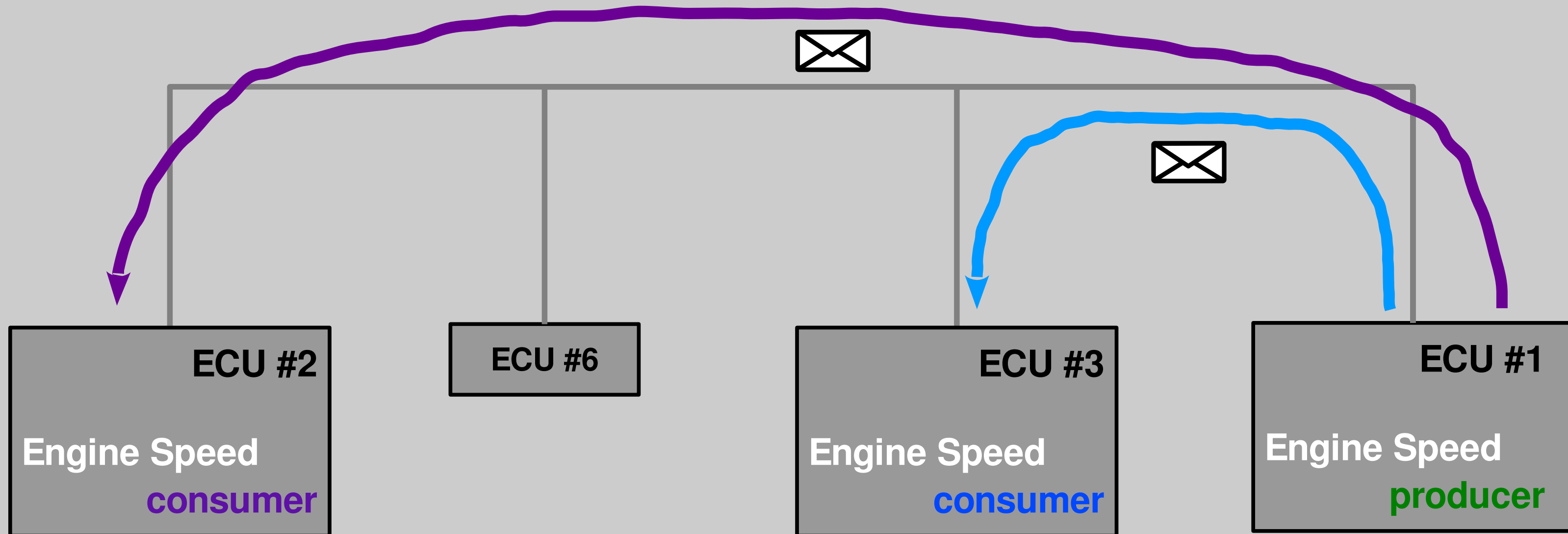
Process #2

Engine speed consumer

With no remote consumers, **engine speed**
is not sent to other ECUs



SCS uses unicast

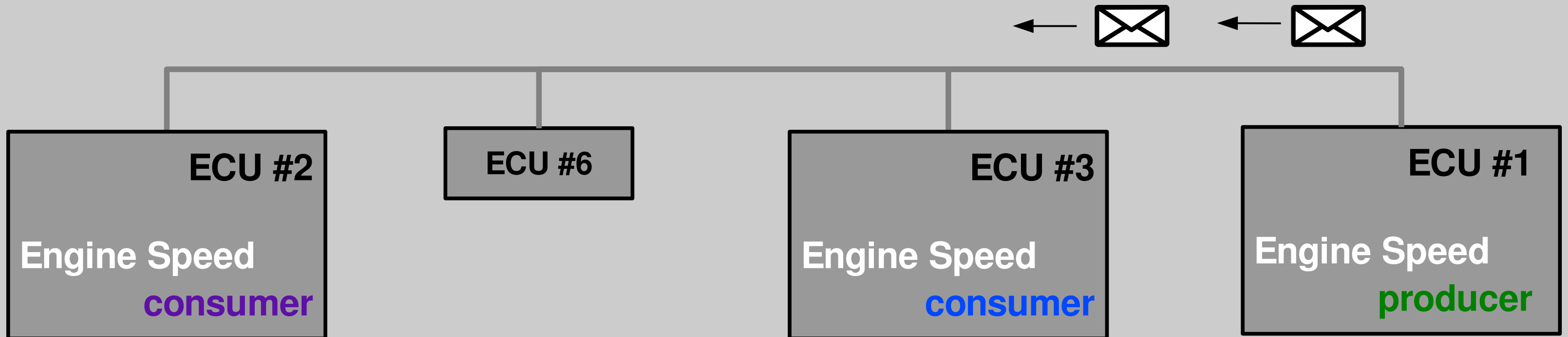


unicast drawback

SCS drawback:

Number of payload messages sent

= Number of ECUs with data consumers



multicast & broadcast

SCS does not support multicast and broadcast

..., yet.

subscription based

Repeated slide

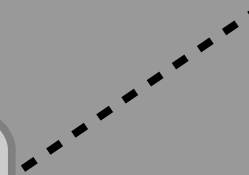
A process needs to **subscribe** to
receive **some data** over **SCS**

Next page

Process #9

ECU #3

SCS
server



Some data

Some data can be any piece of data...

that can be converted to a stream of bytes.

SCS is not aware about the format or contents of the **data** !

... just like any other transport layer.

SCS & a channel

- ◆ SCS only understands channels i.e., channel names.
- ◆ It is up to SCS client to send/receive the actual data over the correct channel.

SCS Client methods

```
class Client
{
    public:
        CommsStatus connect();
        CommsStatus disconnect();

        CommsStatus subscribe(    string channel );
        CommsStatus unsubscribe( string channel );

        CommsStatus sendMsg(    string channel,  string data );
        CommsStatus receiveMsg( string channel,  some buffer )

};
```

receiveMsg

```
class Client
{
    public:
        CommsStatus connect();
        CommsStatus disconnect();

        CommsStatus subscribe(
        CommsStatus unsubscribe

        CommsStatus sendMsg( string channel, string data );
        CommsStatus receiveMsg( string channel, some buffer )
};
```

receiveMsg() gets the whole message
– not just a fragment (aka packet)

duplex at the core

```
class Client
{
    public:
        CommsStatus connect();
        CommsStatus disconnect();

        CommsStatus subscribe(
        CommsStatus unsubscribe( string
                                     ,
        CommsStatus sendMsg( string channel, string data );
        CommsStatus receiveMsg( string channel, some buffer )
};
```

- ◆ **Duplex** operation at its core.
- ◆ However, AMT AF uses SCS in **simplex** mode.

Receive queue size

- ◆ SCS client stores received messages associated with a given channel in a dedicated RX queue.
- ◆ The maximum RX queue size is configurable.
- ◆ Each channel can have a unique receive queue maximum size.
- ◆ The oldest message is discarded when the RX is full and a new message is received.
- ◆ The maximum RX queue size is fixed and cannot be changed at run-time.
- ◆ The only method to modify the RX queue size is to unsubscribe from a channel and re-subscribe it with a different RX queue size.

You've got mail

- ◆ SCS client supports a notification mechanism when a new message (for a channel) has arrived.
- ◆ The notification is on per channel basis.
- ◆ At the time of subscribing to a channel, an application code may provide a pointer to a Condition object.
- ◆ Upon new message arrival, SCS client will set the Condition object.

periodic/random data

Example:

Consumer Process

subscribe to EngSpeedChannel

receiveMsg EngineSpeed

Eng Spd Channel

Producer Process

subscribe to EngSpdOutputCh

sendMsg EngineSpeed



on-request data

Consumer

Request

Response

Producer



on-request data

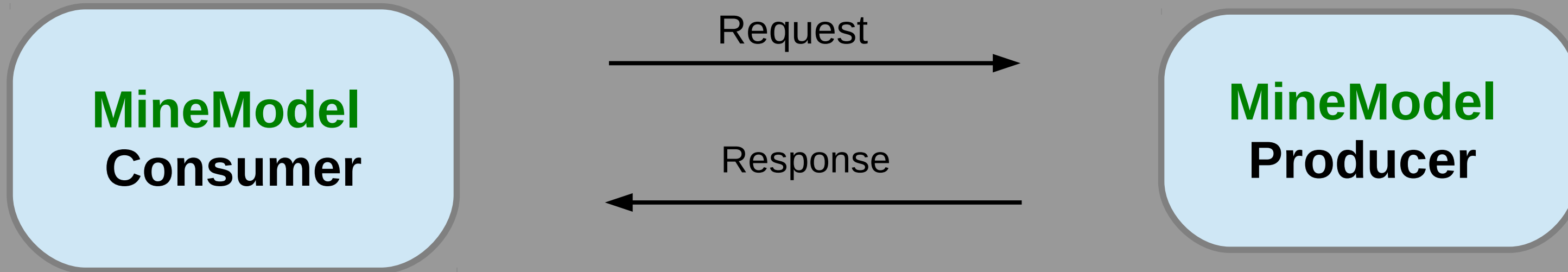
Example:

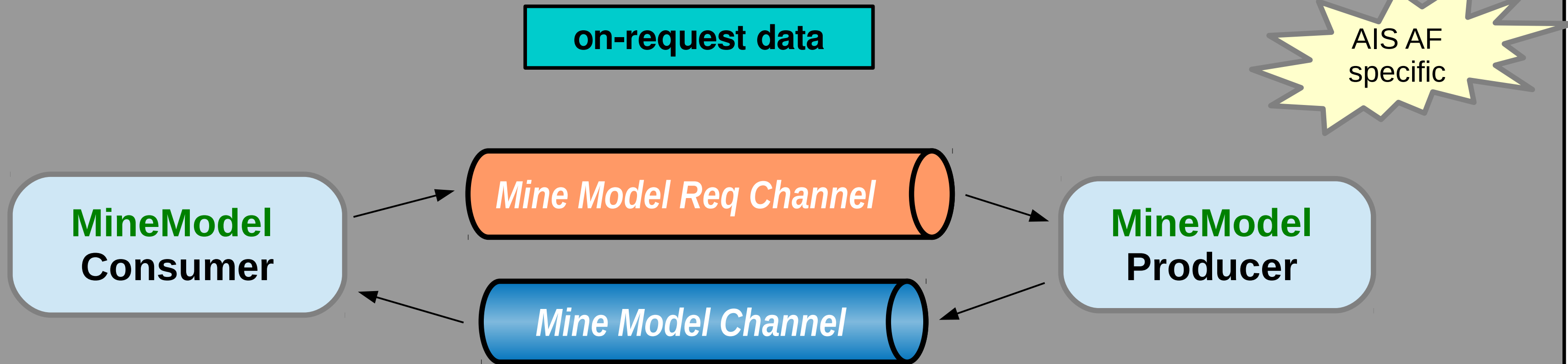
**MineModel
Consumer**

Request

Response

**MineModel
Producer**





In short:

- ◆ **Two SCS channels** are req'd for request & response communication since it is the simplex mode is what AIS AF is build upon.

Some details:

on-request data

AIS AF
specific

MineModel Consumer

subscribe to MineModelReqCh
subscribe to MineModelCh

sendMsg MineModelRequest

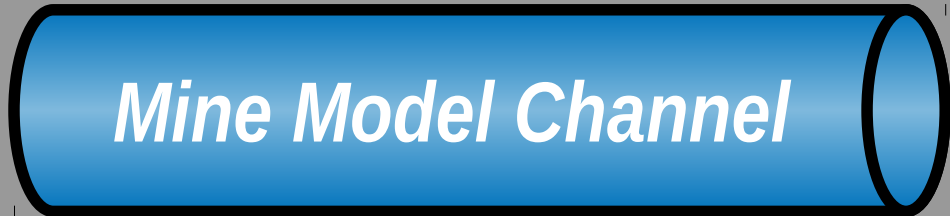
receiveMsg MineModel

MineModel Producer

subscribe to MineModelReqCh
subscribe to MineModelCh

receiveMsg MineModelRequest

sendMsg MineModel



Just F.Y.I

- ◆ An application code leveraging AMT AF is not directly operating on **channels** as this example might suggest.
- ◆ Instead, the application code uses abstract **interfaces** and selecting **interface** implementation is configurable.

MineModel Consumer

subscribe to MineModelReq**Ch**

subscribe to MineModelCh

sendMsg MineModel**Request**

receiveMsg MineModel

Productivity

AMT application Code



AIS Application Framework



SCS

...

as a result AIS AF based application code is compact

MineModel Consumer

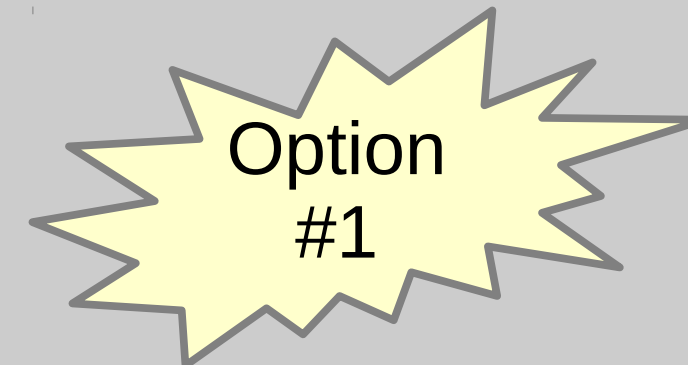
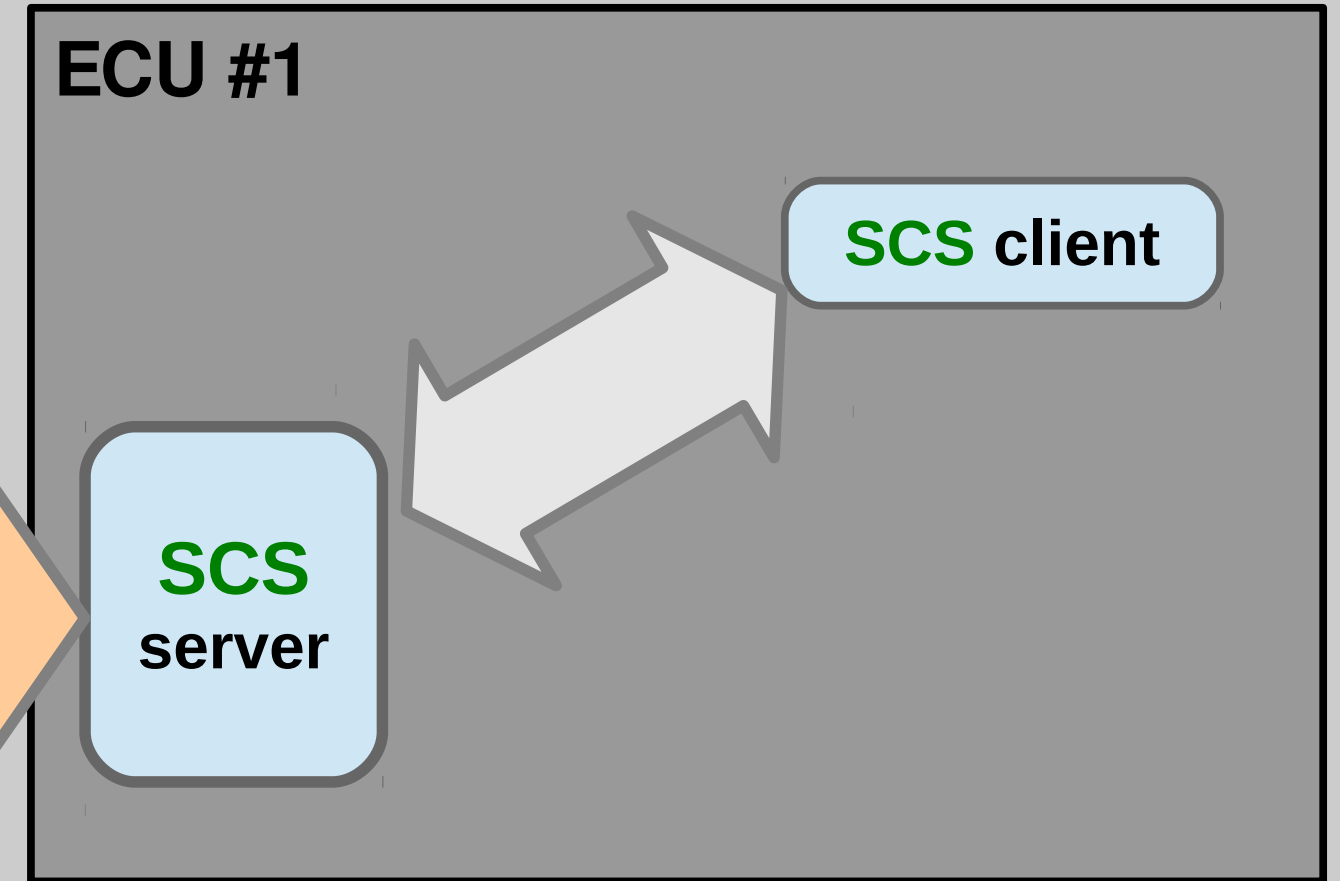
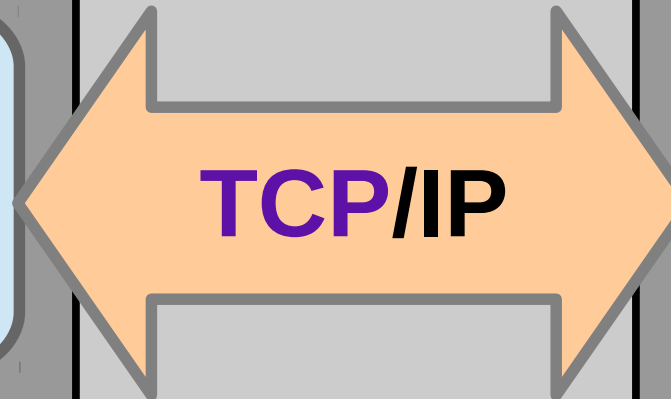
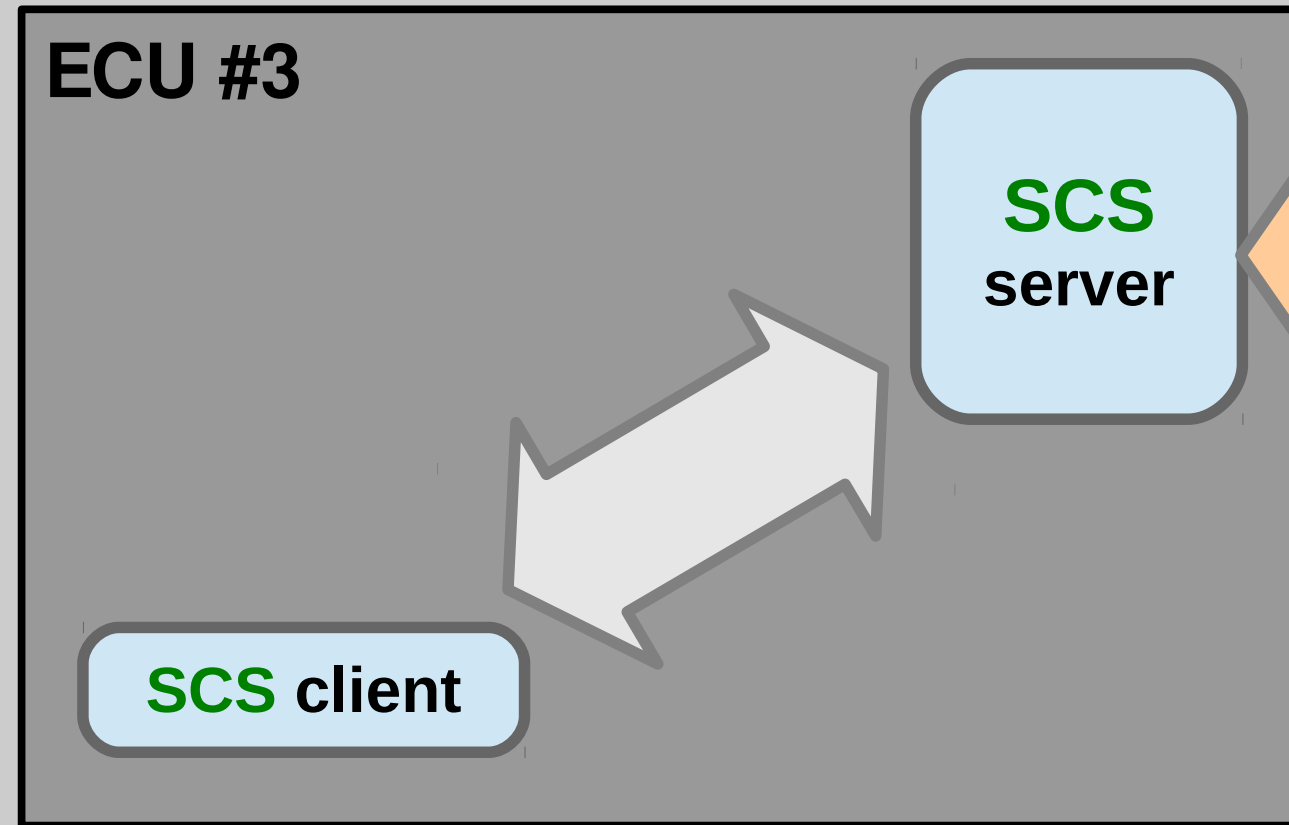
subscribe to MineModelReq**Ch**

subscribe to MineModelCh

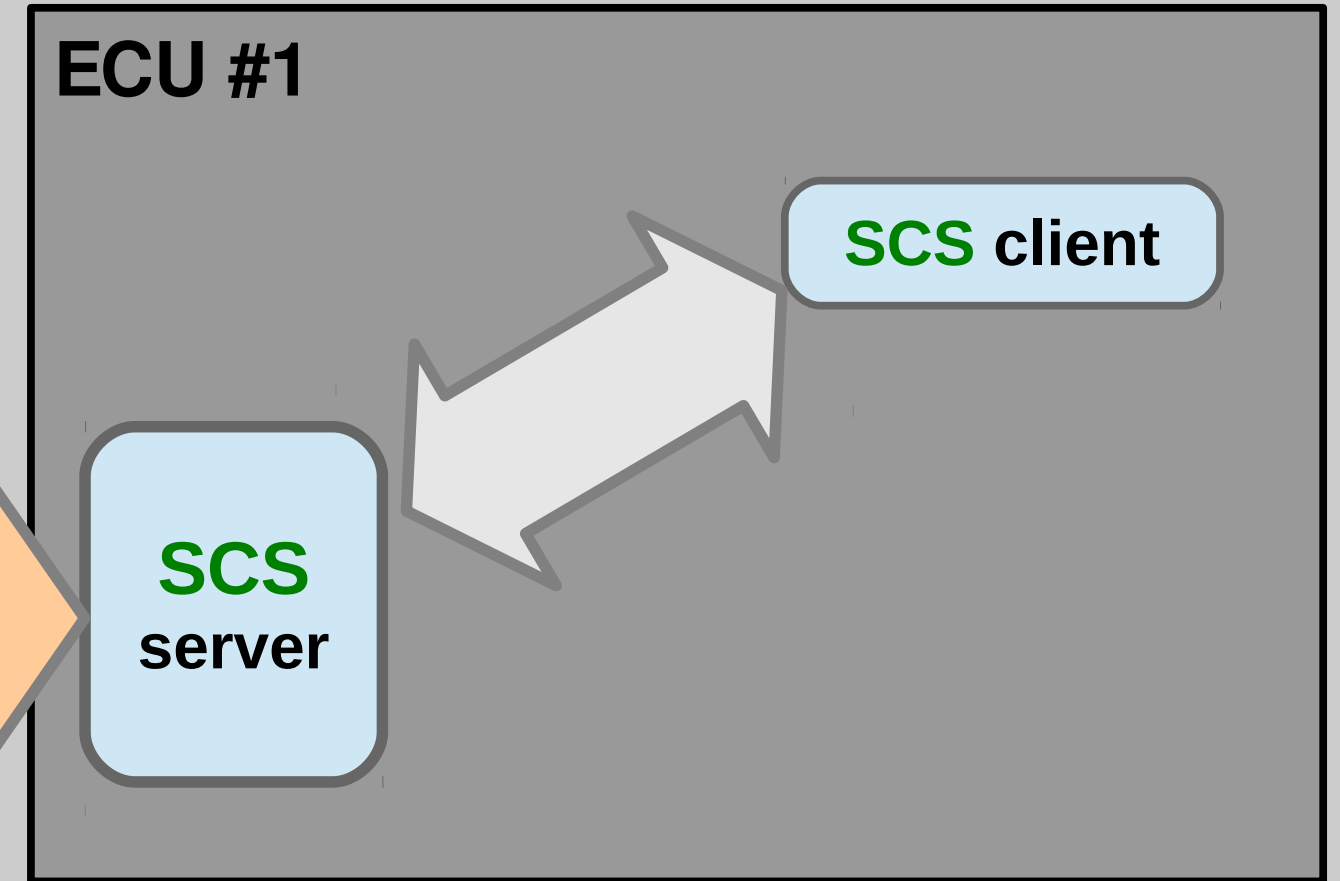
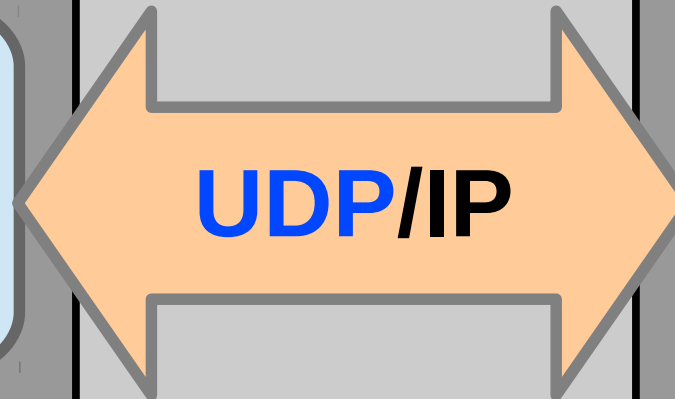
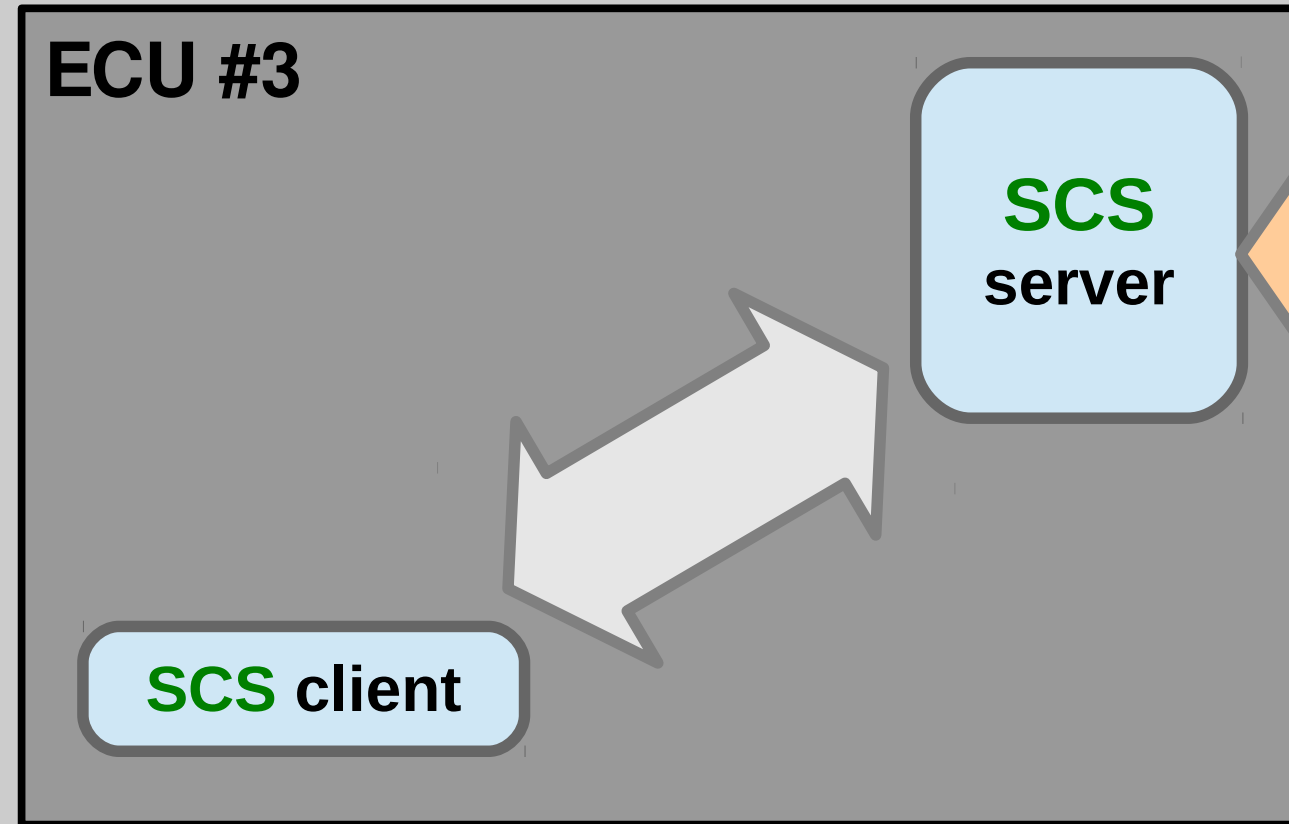
sendMsg MineModel**Request**

receiveMsg MineModel

inter ECU transport



inter ECU transport



Option
#2

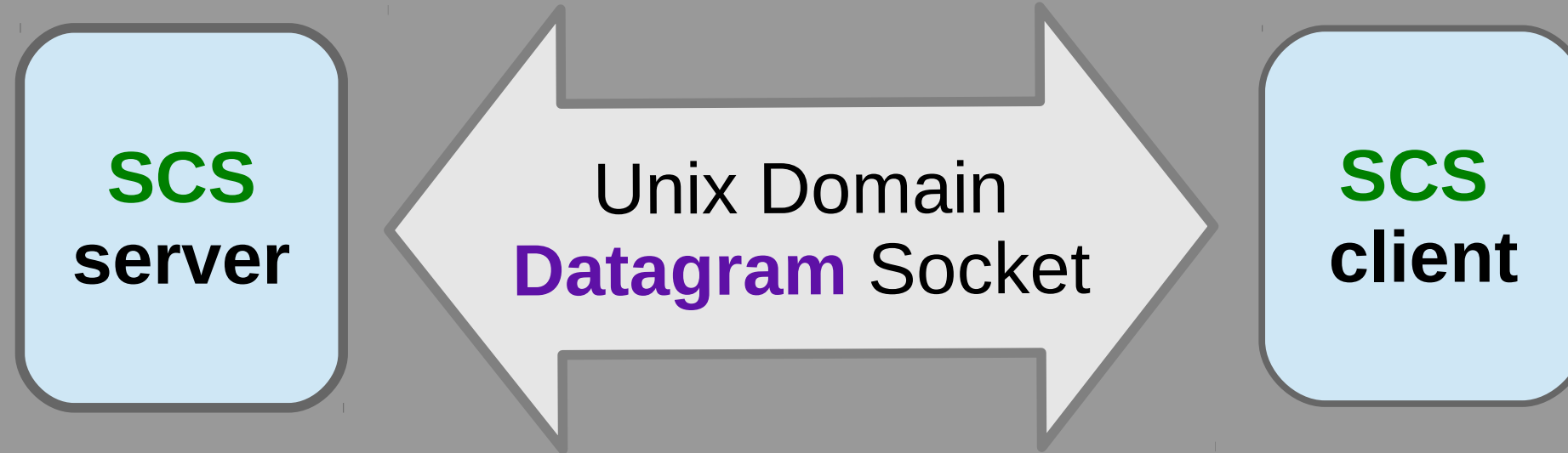
local transport

ECU #1

**SCS
server**

Unix Domain
Datagram Socket

**SCS
client**



local transport – cont'd

ECU #1

SCS
server

Unix Domain
Datagram Socket

SCS
client

- ◆ Fast
- ◆ unreliable – delivery not guaranteed
- ◆ similar to UDP



Other ECUs discovery

- ◆ a SCS server automatically discovers other SCS servers located on other ECUs

Other ECUs discovery

- ◆ a SCS server automatically discovers other SCS servers located on other ECUs
- ◆ The discovery is implemented as a periodic UDP broadcast

Other ECUs discovery

- ◆ a SCS server automatically discovers other SCS servers located on other ECUs
- ◆ The discovery is implemented as a periodic UDP broadcast
- ◆ Only SCS servers on the same subnet can be detected.

Other ECUs discovery

- ◆ a SCS server automatically discovers other SCS servers located on other ECUs
- ◆ The discovery is implemented as a periodic UDP broadcast
- ◆ Only SCS servers on the same subnet can be detected.
- ◆ Dynamic ECU discovery
 - an ECU (running SCS server) joining the network will be detected by other SCS ECUs within 2 seconds
(along with all the channels its SCS clients are producing and consuming)

data priority

- ★ SCS does not support priority on a channel or type of data basis.
- ★ As a result, transferring a huge file will negatively impact message delivery of potentially higher priority content.

priority workaround

A workaround:

- ◆ create two instances of SCS server.
- ◆ One server is dedicated to channels carrying higher priority data
- ◆ The other server is dedicated to channels carrying lower priority data
- ◆ Configure one server to run in higher scheduler priority than the other server

```
char  origin[LEN]; = 64  
char  channel[LEN]; = 64
```

= **128 bytes**

```
uint32_t  length;  
uint32_t  rollingMessageID;  
uint32_t  totalPages;  
uint32_t  pageNumber;  
uint32_t  dataLength;  
uint32_t  headerChecksum;  
uint32_t  dataChecksum;  
timeval   recvMsgTime;
```

= **36 bytes**

SCS overhead

= **164 bytes**

flow control – use cases

Under construction

- ◆ client to server
- ◆ server to client
- ◆ server to server

flow control: client to server

- ◆ SCS server process should be configured to have a higher scheduler priority



Under construction

flow control: server to client

- ◆ SCS server will not overwhelm SCS client (specifically UnixClient)
 - SCS server will not send a package/message to its client (i.e., local client)
that is not ready for I/O operation

implementation - start

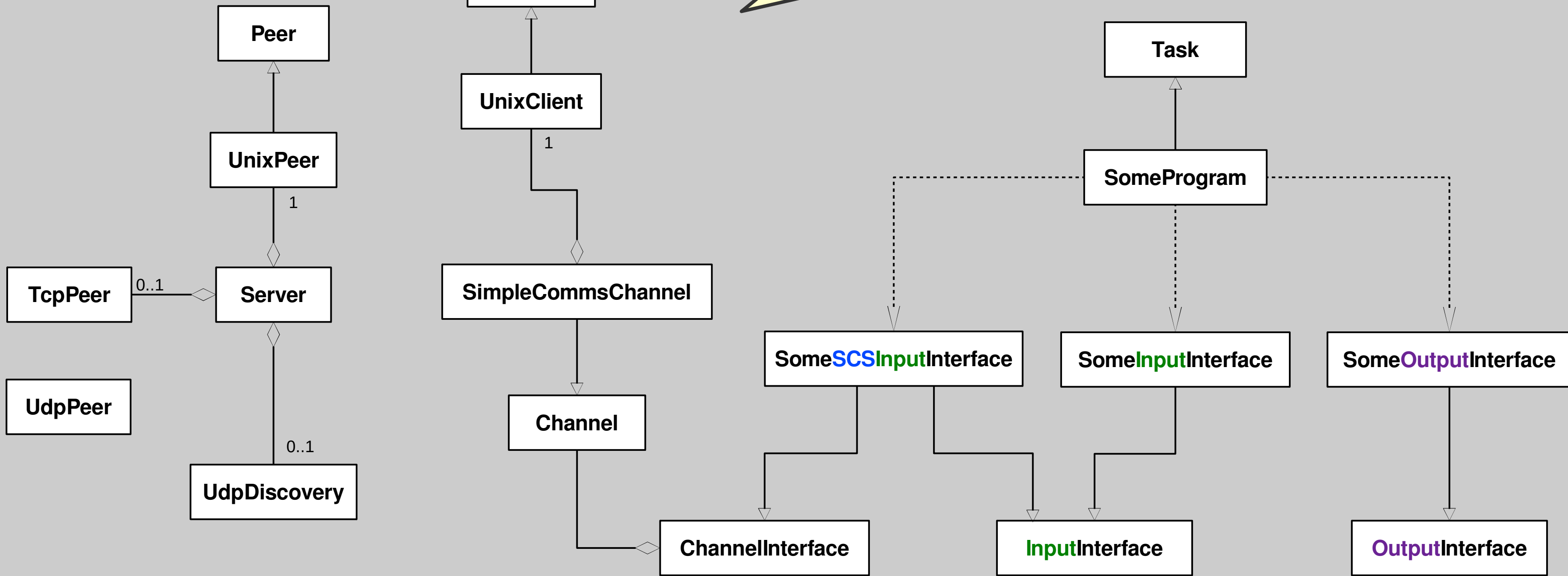
Software layout & implementation details

START

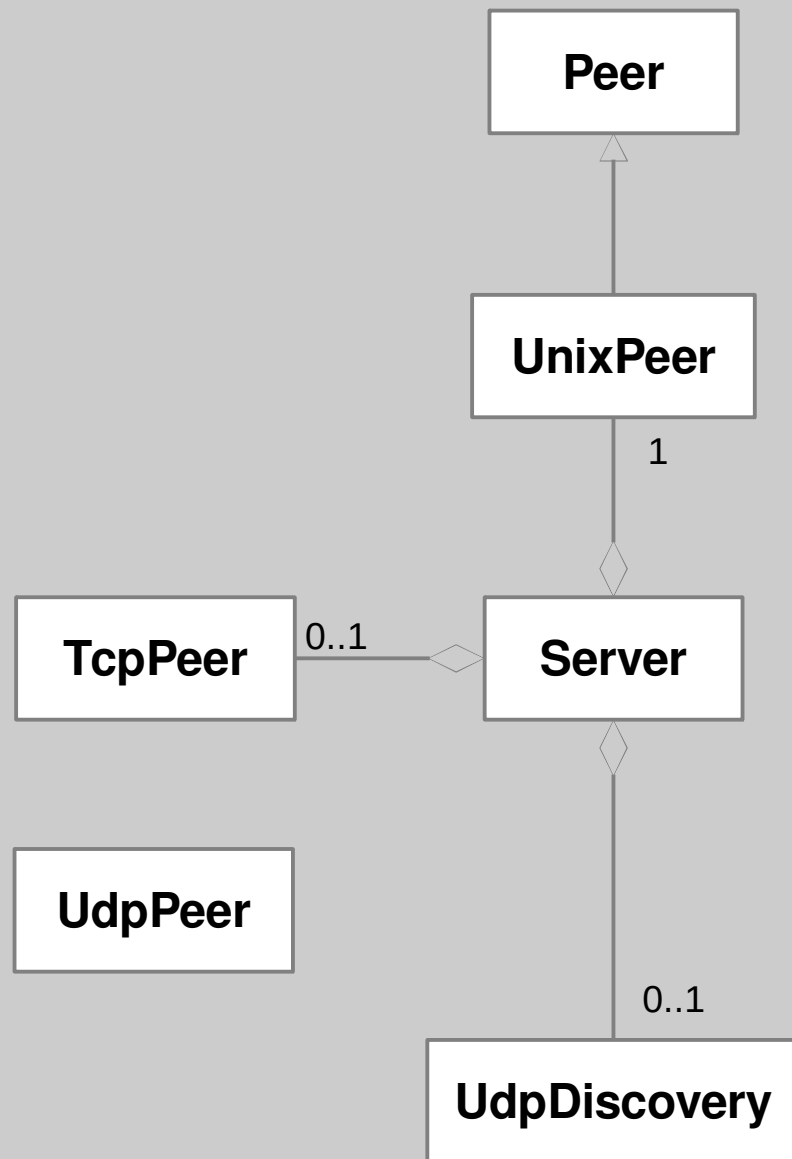
SCS + AF

ECU #1

The high level view

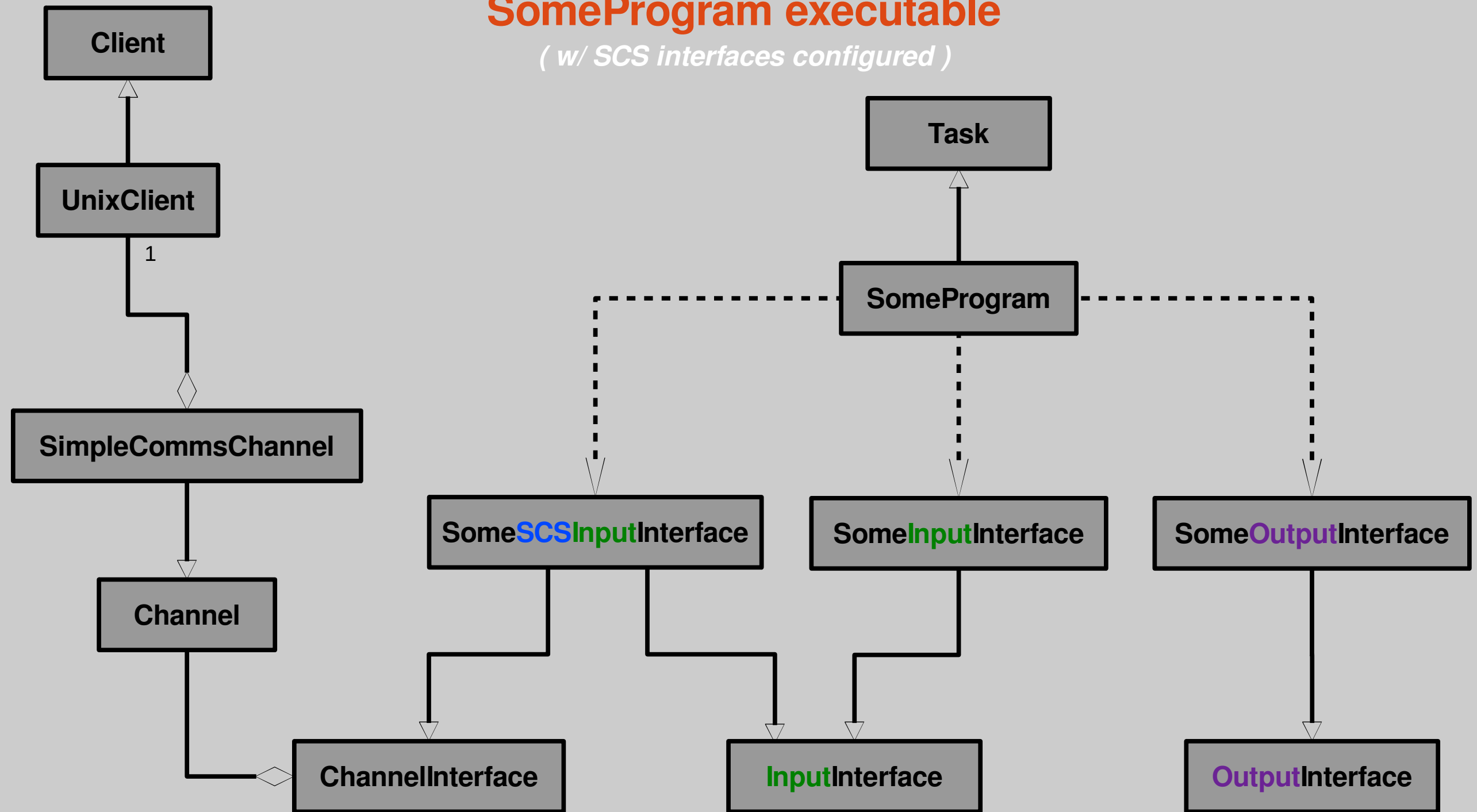


scs + AF other take



SomeProgram executable

(w/ SCS interfaces configured)



SCS

